Regras do machine learning:

Práticas recomendadas para a engenharia de ML

Martin Zinkevich

Este documento tem como objetivo ajudar pessoas com conhecimento básico de machine learning a aproveitar as práticas recomendadas do Google. Ele apresenta um estilo para aprendizado de máquina, semelhante ao Guia de estilo C++ do Google e outros guias conhecidos de programação prática. Se você fez uma aula de machine learning ou criou ou trabalhou em um modelo de machine learning, então você tem o conhecimento necessário para ler este documento.

Rules of ML
Google Open Online Education

Assista no

Martin Zinkevich apresenta 10 das suas regras favoritas de machine learning. Continue lendo para saber mais sobre as 43 regras.

Terminologia

Os termos a seguir vão aparecer repetidamente na nossa discussão sobre o aprendizado de máquina eficaz:

- **Instância**: o que você quer prever. Por exemplo, a instância pode ser uma página da Web que você quer classificar como "sobre gatos" ou "não sobre gatos".
- **Rótulo**: uma resposta para uma tarefa de previsão, seja a resposta produzida por um sistema de aprendizado de máquina ou a resposta correta fornecida nos dados de treinamento. Por exemplo, o rótulo de uma página da Web pode ser "sobre gatos".
- **Recurso**: uma propriedade de uma instância usada em uma tarefa de previsão. Por exemplo, uma página da Web pode ter um recurso "contém a palavra "gato"".
- **Coluna de atributos**: um conjunto de atributos relacionados, como o conjunto de todos os países possíveis em que os usuários podem morar. Um exemplo pode ter um ou mais recursos presentes em uma coluna de atributos. "Coluna de atributos" é uma terminologia específica do Google. Uma coluna de recursos é chamada de "namespace" no sistema VW (no Yahoo/Microsoft) ou de campo (https://www.csie.ntu.edu.tw/%7Ecjlin/libffm/).
- **Exemplo**: uma instância (com os recursos) e um rótulo.
- **Modelo**: uma representação estatística de uma tarefa de previsão. Você treina um modelo com exemplos e, em seguida, usa o modelo para fazer previsões.
- **Métrica**: um número importante. Pode ou não ser otimizada diretamente.
- **Objetivo**: uma métrica que o algoritmo está tentando otimizar.
- **Pipeline**: a infraestrutura que envolve um algoritmo de aprendizado de máquina. Inclui a coleta de dados do front-end, a inserção deles em arquivos de dados de treinamento, o treinamento de um ou mais modelos e a exportação deles para a produção.
- **Taxa de cliques**: a porcentagem de visitantes de uma página da Web que clicam em um link em um anúncio.

Visão geral

Para criar produtos incríveis:

Faça machine learning como o grande engenheiro que você é, não como o grande especialista em machine learning que você não é.

A maioria dos problemas que você vai enfrentar são, na verdade, problemas de engenharia. Mesmo com todos os recursos de um grande especialista em aprendizado de máquina, a maioria dos ganhos vem de recursos excelentes, não de algoritmos de aprendizado de máquina. A abordagem básica é:

1. Confira se o pipeline é sólido de ponta a ponta.
2. Comece com um objetivo razoável.
3. Adicione recursos simples e intuitivos.
4. Confira se o pipeline permanece sólido.

Essa abordagem vai funcionar bem por um longo período. Desvie dessa abordagem apenas quando não houver mais truques simples para avançar. Adicionar complexidade atrasa as versões futuras.

Depois de esgotar os truques simples, o machine learning de ponta pode estar no seu futuro. Consulte a seção sobre projetos de aprendizado de máquina da Fase III (#ml_phase_iii_slowed_growth_optimization_refinement_and_complex_models).

Este documento está organizado da seguinte forma:

1. A primeira parte (#before_machine_learning) ajuda você a entender se é o momento certo para criar um sistema de aprendizado de máquina.
2. A segunda parte (#ml_phase_i_your_first_pipeline) é sobre a implantação do seu primeiro pipeline.
3. A terceira parte (#ml_phase_ii_feature_engineering) trata do lançamento e da iteração, além de adicionar novos recursos ao pipeline, como avaliar modelos e o desvio de treinamento/exibição.
4. A parte final (#ml_phase_iii_slowed_growth_optimization_refinement_and_complex_models) explica o que fazer quando você atinge um platô.
5. Em seguida, há uma lista de trabalhos relacionados (#related_work) e um apêndice (#appendix) com alguns antecedentes sobre os sistemas usados com frequência como exemplos neste documento.

Antes do aprendizado de máquina

Regra nº 1: não tenha medo de lançar um produto sem o aprendizado de máquina.

O machine learning é legal, mas exige dados. Teoricamente, é possível usar dados de um problema diferente e ajustar o modelo para um novo produto, mas isso provavelmente terá um desempenho inferior às **heurísticas** (https://developers.google.com/machine-learning/glossary?hl=pt-br#heuristic) básicas. Se você acha que o machine learning vai dar um impulso de 100%, uma heurística vai dar 50% do caminho.

Por exemplo, se você estiver classificando apps em um app marketplace, use a taxa de instalação ou o número de instalações como heurísticas. Se você detectar spam, filtre os editores que já enviaram spam. Não tenha medo de usar a edição humana. Se você precisar classificar os contatos, classifique o mais usado recentemente com a maior pontuação ou até mesmo em ordem alfabética. Se o aprendizado de máquina não for absolutamente necessário para seu produto, não o use até ter dados.

Regra 2: primeiro, projete e implemente métricas.

Antes de formalizar o que seu sistema de aprendizado de máquina vai fazer, acompanhe o máximo possível no seu sistema atual. Faça isso pelos seguintes motivos:

1. É mais fácil conseguir a permissão dos usuários do sistema com antecedência.
2. Se você acha que algo pode ser um problema no futuro, é melhor extrair dados históricos agora.
3. Se você projetar seu sistema pensando na instrumentação de métricas, as coisas vão melhorar para você no futuro. Especificamente, não procure strings nos registros para instrumentar suas métricas.
4. Você vai notar o que mudou e o que permaneceu igual. Por exemplo, suponha que você queira otimizar diretamente os usuários ativos por um dia. No entanto, durante as primeiras manipulações do sistema, você pode notar que alterações drásticas na experiência do usuário não mudam essa métrica de forma perceptível.

A equipe do Google Plus (#google_plus_overview) mede as leituras, os novos compartilhamentos por leitura, as marcações "Gostei" por leitura, os comentários/leitura, os comentários por usuário, os novos compartilhamentos por usuário etc., que são usados para calcular a qualidade de uma postagem no momento da veiculação. **Além disso, é importante ter um**

framework de experimentos, em que você pode agrupar usuários em grupos e agregar estatísticas por experimento. Consulte Regra 12 (#rule_12_don_t_overthink_which_objective_you_choose_to_directly_optimize).

Ao coletar métricas de forma mais liberal, você pode ter uma visão mais ampla do seu sistema. Notou algum problema? Adicione uma métrica para monitorar. Está animado com alguma mudança quantitativa na última versão? Adicione uma métrica para monitorar.

Regra 3: escolha o aprendizado de máquina em vez de uma heurística complexa.

Uma heurística simples pode fazer com que seu produto seja lançado. Uma heurística complexa não pode ser mantida. Depois de ter dados e uma ideia básica do que você está tentando fazer, passe para o aprendizado de máquina. Como na maioria das tarefas de engenharia de software, é importante atualizar constantemente sua abordagem, seja ela heurística ou de aprendizado de máquina. O modelo de aprendizado de máquina é mais fácil de atualizar e manter (consulte a Regra 16 (#rule_16_plan_to_launch_and_iterate)).

Fase I de ML: seu primeiro pipeline

Concentre-se na infraestrutura do sistema para o primeiro pipeline. É divertido pensar em todo o aprendizado de máquina imaginativo que você vai fazer, mas será difícil descobrir o que está acontecendo se você não confiar primeiro no pipeline.

Regra 4: mantenha o primeiro modelo simples e configure a infraestrutura corretamente.

O primeiro modelo oferece o maior impulso ao seu produto, então ele não precisa ser sofisticado. Mas você vai encontrar muitos mais problemas de infraestrutura do que espera. Antes que alguém possa usar seu novo sistema de aprendizado de máquina, você precisa determinar:

- Como conseguir exemplos para seu algoritmo de aprendizado.
- Uma primeira definição do que "bom" e "ruim" significam para o sistema.
- Como integrar o modelo ao seu aplicativo. Você pode aplicar o modelo em tempo real ou pré-calcular o modelo em exemplos off-line e armazenar os resultados em uma tabela. Por exemplo, você pode pré-classificar páginas da Web e armazenar os resultados em uma tabela, mas pode ser necessário classificar as mensagens de chat em tempo real.

A escolha de recursos simples facilita a garantia de que:

- Os recursos chegam ao algoritmo de aprendizado corretamente.
- O modelo aprende pesos razoáveis.
- Os recursos chegam ao modelo no servidor corretamente.

Quando você tem um sistema que faz essas três coisas de maneira confiável, você fez a maior parte do trabalho. O modelo simples fornece métricas de referência e um comportamento de referência que você pode usar para testar modelos mais complexos. Algumas equipes têm como objetivo uma primeira versão "neutra": uma primeira versão que prioriza explicitamente os ganhos de aprendizado de máquina para evitar distrações.

Regra 5: teste a infraestrutura de forma independente do machine learning.

Verifique se a infraestrutura pode ser testada e se as partes de aprendizado do sistema estão encapsuladas para que você possa testar tudo relacionado a elas. Especificamente:

1. Teste a transferência de dados para o algoritmo. Verifique se as colunas de elementos que precisam ser preenchidas foram preenchidas. Quando a privacidade permitir, inspecione manualmente a entrada do algoritmo de treinamento. Se possível, verifique as estatísticas no pipeline em comparação com as estatísticas dos mesmos dados processados em outro lugar.
2. Teste a extração de modelos do algoritmo de treinamento. Verifique se o modelo no ambiente de treinamento tem a mesma pontuação que o modelo no ambiente de veiculação (consulte a Regra 37 (#rule_37_measure_training_serving_skew)).

O aprendizado de máquina tem um elemento de imprevisibilidade. Portanto, verifique se você tem testes para o código de criação de exemplos no treinamento e na exibição e se é possível carregar e usar um modelo fixo durante a exibição. Além disso, é importante entender seus dados: consulte Conselhos práticos para análise de conjuntos de dados grandes e complexos (http://www.unofficialgoogledatascience.com/2016/10/practical-advice-for-analysis-of-large.html).

Regra 6: tenha cuidado com os dados descartados ao copiar pipelines.

Muitas vezes, criamos um pipeline copiando um pipeline existente (ou seja, programação cargo cult (https://wikipedia.org/wiki/Cargo_cult_programming)).), e o pipeline antigo descarta dados necessários para o novo. Por exemplo, o pipeline do Google Plus (#google_plus_overview)

What's Hot elimina as postagens mais antigas porque ele tenta classificar as postagens mais recentes. Esse pipeline foi copiado para uso na transmissão do Google Plus (#google_plus_overview), em que as postagens mais antigas ainda são significativas, mas o pipeline ainda estava excluindo postagens antigas. Outro padrão comum é registrar apenas os dados que foram vistos pelo usuário. Portanto, esses dados são inúteis se quisermos modelar por que uma postagem específica não foi vista pelo usuário, porque todos os exemplos negativos foram descartados. Ocorreu um problema semelhante no Google Play. Durante o trabalho na página inicial de apps da Play, um novo pipeline foi criado, que também continha exemplos da página de destino do Play Games sem nenhum recurso para eliminar a ambiguidade de onde cada exemplo veio.

Regra 7: transforme heurísticas em recursos ou processe-as externamente.

Normalmente, os problemas que o aprendizado de máquina está tentando resolver não são completamente novos. Há um sistema para classificar, ou classificar, ou qualquer problema que você está tentando resolver. Isso significa que há várias regras e heurísticas. **Essas mesmas heurísticas podem ajudar quando ajustadas com o aprendizado de máquina.** Suas heurísticas precisam ser extraídas para qualquer informação que elas tenham, por dois motivos. Primeiro, a transição para um sistema de aprendizado de máquina será mais suave. Em segundo lugar, essas regras geralmente contêm muita intuição sobre o sistema que você não quer descartar. Há quatro maneiras de usar uma heurística:

- Pré-processe usando a heurística. Se o recurso for incrível, essa é uma opção. Por exemplo, se, em um filtro de spam, o remetente já estiver na lista de bloqueio, não tente reaprender o que "lista de bloqueio" significa. Bloqueie a mensagem. Essa abordagem faz mais sentido em tarefas de classificação binária.
- Crie um recurso. Criar um recurso diretamente da heurística é ótimo. Por exemplo, se você usar uma heurística para calcular uma pontuação de relevância para um resultado de consulta, poderá incluir a pontuação como o valor de um recurso. Mais tarde, você pode usar técnicas de aprendizado de máquina para ajustar o valor (por exemplo, convertendo o valor em um de um conjunto finito de valores discretos ou combinando-o com outros recursos), mas comece usando o valor bruto produzido pela heurística.
- Extraia as entradas brutas da heurística. Se houver uma heurística para apps que combine o número de instalações, o número de caracteres no texto e o dia da semana, separe essas partes e alimente essas entradas na aprendizagem separadamente. Algumas técnicas que se aplicam a conjuntos também se aplicam aqui (consulte a Regra 40 (#rule_40_keep_ensembles_simple)).

- Modifique o rótulo. Essa é uma opção quando você acha que a heurística captura informações que não estão no rótulo. Por exemplo, se você está tentando maximizar o número de downloads, mas também quer conteúdo de qualidade, talvez a solução seja multiplicar o rótulo pelo número médio de estrelas que o app recebeu. Há muita margem de manobra aqui. Consulte "Seu primeiro objetivo" (#your_first_objective).

Tenha cuidado com a complexidade adicional ao usar heurísticas em um sistema de ML. O uso de heurísticas antigas no novo algoritmo de aprendizado de máquina pode ajudar a criar uma transição suave, mas pense se há uma maneira mais simples de conseguir o mesmo efeito.

Monitoramento

Em geral, pratique bons hábitos de alerta, como tornar os alertas úteis e ter uma página de painel.

Regra 8: conheça os requisitos de atualização do seu sistema.

Quanto a performance é degradada se você tiver um modelo com um dia de idade? Uma semana de idade? Um quarto de idade? Essas informações podem ajudar você a entender as prioridades da sua monitoração. Se você perder a qualidade do produto significativamente se o modelo não for atualizado por um dia, faz sentido ter um engenheiro que o monitore continuamente. A maioria dos sistemas de veiculação de anúncios tem novos anúncios para processar todos os dias e precisa ser atualizada diariamente. Por exemplo, se o modelo de ML para a Pesquisa do Google Play (#google_play_overview) não for atualizado, isso poderá ter um impacto negativo em menos de um mês. Alguns modelos do recurso O que está em alta no Google Plus (#google_plus_overview) não têm um identificador de postagem no modelo, para que eles possam exportar esses modelos com pouca frequência. Outros modelos que têm identificadores de postagem são atualizados com muito mais frequência. Além disso, a atualidade pode mudar ao longo do tempo, principalmente quando as colunas de atributos são adicionadas ou removidas do modelo.

Regra 9: detectar problemas antes de exportar modelos.

Muitos sistemas de machine learning têm uma etapa em que você exporta o modelo para ser veiculado. Se houver um problema com um modelo exportado, ele será um problema de usuário.

Faça verificações de sanidade antes de exportar o modelo. Especifique se o desempenho do modelo é razoável nos dados retidos. Ou, se você tiver dúvidas sobre os dados, não exporte um modelo. Muitas equipes que implantam modelos continuamente verificam a área sob a curva ROC (https://wikipedia.org/wiki/Receiver_operating_characteristic) (ou AUC) antes da exportação. **Problemas com modelos que não foram exportados exigem um alerta por e-mail, mas problemas em um modelo voltado ao usuário podem exigir uma página.** Por isso, é melhor esperar e ter certeza antes de afetar os usuários.

Regra nº 10: fique de olho em falhas silenciosas.

Esse problema ocorre mais em sistemas de aprendizado de máquina do que em outros tipos de sistemas. Suponha que uma tabela específica que está sendo mesclada não esteja mais sendo atualizada. O sistema de aprendizado de máquina vai se ajustar, e o comportamento vai continuar razoavelmente bom, diminuindo gradualmente. Às vezes, você encontra tabelas de meses atrás, e uma atualização simples melhora a performance mais do que qualquer outro lançamento no trimestre. A cobertura de um recurso pode mudar devido a mudanças na implementação. Por exemplo, uma coluna de atributos pode ser preenchida em 90% dos exemplos e, de repente, cair para 60% dos exemplos. O Play uma vez tinha uma tabela desatualizada há seis meses, e a atualização dela sozinha aumentou a taxa de instalação em 2%. Se você acompanhar as estatísticas dos dados e inspecionar os dados manualmente, poderá reduzir esse tipo de falha.

Regra 11: forneça proprietários e documentação às colunas de atributos.

Se o sistema for grande e tiver muitas colunas de recursos, saiba quem criou ou está mantendo cada coluna de recursos. Se a pessoa que entende uma coluna de recurso estiver saindo, verifique se alguém tem as informações. Embora muitas colunas de recursos tenham nomes descritivos, é bom ter uma descrição mais detalhada do que é o recurso, de onde ele veio e como ele pode ajudar.

Seu primeiro objetivo

Você tem muitas métricas ou medições sobre o sistema que interessa, mas o algoritmo de aprendizado de máquina geralmente exige um único **objetivo, um número que o algoritmo está "tentando" otimizar**. Aqui, faço uma distinção entre objetivos e métricas: uma métrica é qualquer número que seu sistema informa, o que pode ou não ser importante. Consulte também Regra 2 (#rule_2_first_design_and_implement_metrics).

Regra 12: não pense demais sobre qual objetivo você escolhe otimizar diretamente.

Você quer ganhar dinheiro, deixar seus usuários felizes e tornar o mundo melhor. Há muitas métricas importantes, e você precisa medir todas elas (consulte a Regra 2 (#rule_2_first_design_and_implement_metrics)). No entanto, no início do processo de aprendizado de máquina, você vai notar que todos eles vão aumentar, mesmo aqueles que você não otimiza diretamente. Por exemplo, suponha que você se importe com o número de cliques e o tempo gasto no site. Se você otimizar para o número de cliques, é provável que o tempo gasto aumente.

Portanto, mantenha a simplicidade e não se preocupe muito em equilibrar métricas diferentes quando você ainda pode aumentar todas elas com facilidade. Não leve essa regra muito a sério: não confunda seu objetivo com a saúde final do sistema (consulte a Regra 39 (#rule_39_launch_decisions_are_a_proxy_for_long_term_product_goals)). **Se você aumentar a métrica otimizada diretamente, mas decidir não lançar, talvez seja necessário fazer uma revisão objetiva.**

Regra 13: escolha uma métrica simples, observável e atribuível para sua primeira meta.

Muitas vezes, você não sabe qual é o objetivo real. Você acha que sabe, mas, ao analisar os dados e a análise lado a lado do sistema antigo e do novo sistema de ML, você percebe que quer ajustar o objetivo. Além disso, diferentes membros da equipe muitas vezes não concordam com o objetivo real. **O objetivo de ML precisa ser fácil de medir e ser um proxy para o objetivo "verdadeiro".** Na verdade, muitas vezes não há um objetivo "verdadeiro" (consulte Regra 39 (#rule_39_launch_decisions_are_a_proxy_for_long_term_product_goals)). Então, treine com o objetivo simples de ML e considere ter uma "camada de política" que permita adicionar outra lógica (provavelmente muito simples) para fazer a classificação final.

O mais fácil de modelar é um comportamento do usuário que é diretamente observado e atribuível a uma ação do sistema:

- O link classificado foi clicado?
- O objeto classificado foi transferido por download?
- O objeto classificado foi encaminhado/respondido/enviado por e-mail?
- Esse objeto classificado foi avaliado?
- O objeto mostrado foi marcado como spam/pornografia/ofensivo?

Evite modelar efeitos indiretos no início:

- O usuário visitou no dia seguinte?
- Por quanto tempo o usuário visitou o site?
- Quantos usuários ativos por dia você teve?

Os efeitos indiretos geram métricas excelentes e podem ser usados durante testes A/B e decisões de lançamento.

Por fim, não tente fazer o machine learning descobrir:

- O usuário está satisfeito com o produto?
- O usuário está satisfeito com a experiência?
- O produto está melhorando o bem-estar geral do usuário?
- Como isso afetará a saúde geral da empresa?

Todos esses são importantes, mas também são muito difíceis de medir. Em vez disso, use proxies: se o usuário estiver satisfeito, ele vai permanecer no site por mais tempo. Se o usuário estiver satisfeito, ele vai voltar amanhã. No que diz respeito ao bem-estar e à saúde da empresa, o julgamento humano é necessário para conectar qualquer objetivo aprendido pela máquina à natureza do produto que você está vendendo e ao seu plano de negócios.

Regra 14: começar com um modelo interpretável facilita a depuração.

A regressão linear, a logística e a Poisson são motivadas diretamente por um modelo probabilístico. Cada previsão pode ser interpretada como uma probabilidade ou um valor esperado. Isso facilita a depuração em relação a modelos que usam objetivos (perda zero-um, várias perdas de articulação e assim por diante) que tentam otimizar diretamente a precisão da classificação ou a performance da classificação. Por exemplo, se as probabilidades no treinamento se desviarem das probabilidades previstas em comparações lado a lado ou ao inspecionar o sistema de produção, esse desvio poderá revelar um problema.

Por exemplo, na regressão linear, logística ou de Poisson, **há subconjuntos de dados em que a expectativa média prevista é igual ao rótulo médio (calibrado no momento 1 ou apenas calibrado)**. Isso é verdade, supondo que você não tenha regularização e que seu algoritmo tenha convergido, e é aproximadamente verdade em geral. Se você tiver um atributo que é 1 ou 0 para cada exemplo, o conjunto de três exemplos em que esse atributo é 1 será calibrado.

Além disso, se você tiver um recurso que é 1 para cada exemplo, o conjunto de todos os exemplos será calibrado.

Com modelos simples, é mais fácil lidar com loops de feedback (consulte a Regra 36 (#rule_36_avoid_feedback_loops_with_positional_features)). Muitas vezes, usamos essas previsões probabilísticas para tomar uma decisão, por exemplo, classificar postagens em valor esperado decrescente (ou seja, probabilidade de clique/download/etc.). **No entanto, lembre-se de que, na hora de escolher qual modelo usar, a decisão é mais importante do que a probabilidade de os dados serem fornecidos ao modelo (consulte a Regra 27** (#rule_27_try_to_quantify_observed_undesirable_behavior)**).**

Regra 15: separe a filtragem de spam e a classificação de qualidade em uma camada de política.

A classificação de qualidade é uma arte, mas a filtragem de spam é uma guerra. Os indicadores que você usa para determinar posts de alta qualidade vão ficar óbvios para quem usa seu sistema, e eles vão ajustar as postagens para ter essas propriedades. Portanto, sua classificação de qualidade deve se concentrar em classificar o conteúdo postado de boa fé. Não desconsidere o Aprendizado de classificação de qualidade para classificar o spam com uma nota alta. **Da mesma forma, o conteúdo "ousado" precisa ser tratado separadamente do Ranking de qualidade.** O filtro de spam é uma história diferente. É esperado que os recursos que você precisa gerar mudem constantemente. Muitas vezes, há regras óbvias que você coloca no sistema (se uma postagem tiver mais de três votos de spam, não a recupere, e assim por diante). Qualquer modelo aprendido precisa ser atualizado diariamente, se não mais rápido. A reputação do criador do conteúdo será muito importante.

Em algum nível, a saída desses dois sistemas precisa ser integrada. Tenha em mente que a filtragem de spam nos resultados da pesquisa provavelmente é mais rigorosa do que a filtragem de spam em mensagens de e-mail. Além disso, é uma prática padrão remover spam dos dados de treinamento do classificador de qualidade.

Fase II do ML: engenharia de atributos

Na primeira fase do ciclo de vida de um sistema de aprendizado de máquina, as questões importantes são inserir os dados de treinamento no sistema de aprendizado, instrumentar as métricas de interesse e criar uma infraestrutura de veiculação. **Depois que você tiver um**

sistema de ponta a ponta funcional com testes de unidade e de sistema instrumentados, a Fase II vai começar.

Na segunda fase, há muitas frutas fáceis de colher. Há uma variedade de recursos óbvios que podem ser incluídos no sistema. Assim, a segunda fase do aprendizado de máquina envolve extrair o máximo de recursos possível e combinar de maneira intuitiva. Durante essa fase, todas as métricas ainda devem estar aumentando. Haverá muitos lançamentos, e é um ótimo momento para atrair muitos engenheiros que podem combinar todos os dados necessários para criar um sistema de aprendizado realmente incrível.

Regra 16: planeje o lançamento e a iteração.

Não espere que o modelo em que você está trabalhando agora seja o último que você vai lançar ou que você vai parar de lançar modelos. Portanto, considere se a complexidade que você está adicionando com esse lançamento vai atrasar lançamentos futuros. Muitas equipes lançaram um modelo por trimestre ou mais por anos. Há três motivos básicos para lançar novos modelos:

- Você está criando novos recursos.
- Você está ajustando a regularização e combinando recursos antigos de novas maneiras.
- Você está ajustando o objetivo.

De qualquer forma, dar um pouco de atenção a um modelo pode ser bom: analisar os dados alimentados no exemplo pode ajudar a encontrar novos sinais, bem como os antigos e quebrados. Ao criar seu modelo, pense em como é fácil adicionar, remover ou recombinar recursos. Pense em como é fácil criar uma nova cópia do pipeline e verificar se ele está correto. Pense se é possível ter duas ou três cópias em execução em paralelo. Por fim, não se preocupe se o recurso 16 de 35 vai entrar nessa versão do pipeline. Você vai receber no próximo trimestre.

Regra 17: comece com recursos observados e informados diretamente, em vez de recursos aprendidos.

Isso pode ser um ponto polêmico, mas evita muitas armadilhas. Primeiro, vamos descrever o que é um recurso aprendido. Ele é um recurso gerado por um sistema externo (como um sistema de agrupamento não supervisionado) ou pelo próprio aprendiz (por exemplo, usando um modelo fatorado ou aprendizado profundo). Elas podem ser úteis, mas podem ter muitos problemas. Por isso, elas não devem estar no primeiro modelo.

Se você usa um sistema externo para criar um recurso, lembre-se de que o sistema externo tem o próprio objetivo. O objetivo do sistema externo pode estar apenas fracamente correlacionado ao seu objetivo atual. Se você tirar um snapshot do sistema externo, ele poderá ficar desatualizado. Se você atualizar os recursos do sistema externo, os significados poderão mudar. Se você usa um sistema externo para fornecer um recurso, saiba que essa abordagem exige muito cuidado.

O principal problema com modelos fatorados e modelos profundos é que eles não são convexos. Portanto, não há garantia de que uma solução ideal possa ser aproximada ou encontrada, e os mínimos locais encontrados em cada iteração podem ser diferentes. Essa variação dificulta a avaliação se o impacto de uma mudança no sistema é significativo ou aleatório. Ao criar um modelo sem recursos avançados, você pode ter uma excelente performance de referência. Depois que esse valor de referência for alcançado, você poderá tentar abordagens mais esotéricas.

Regra 18: explorar com recursos de conteúdo que se generalizam em todos os contextos.

Muitas vezes, um sistema de machine learning é uma pequena parte de um cenário muito maior. Por exemplo, se você imaginar uma postagem que pode ser usada no "O que está bombando", muitas pessoas vão marcar com +1, compartilhar de novo ou comentar uma postagem antes mesmo que ela seja mostrada no "O que está bombando". Se você fornecer essas estatísticas ao aluno, ele poderá promover novas postagens que não têm dados no contexto em que está otimizando. A seção "Assistir a seguir" do YouTube (#youtube_overview) pode usar o número de visualizações ou co- visualizações (contagens de quantas vezes um vídeo foi assistido depois de outro) da pesquisa do YouTube (#youtube_overview). Também é possível usar classificações explícitas do usuário. Por fim, se você tiver uma ação do usuário que está usando como um rótulo, ver essa ação no documento em um contexto diferente pode ser um ótimo recurso. Todos esses recursos permitem que você inclua novos conteúdos no contexto. Não se trata de personalização: descubra se alguém gosta do conteúdo nesse contexto primeiro e, em seguida, descubra quem gosta mais ou menos.

Regra 19: use recursos muito específicos quando possível.

Com muitos dados, é mais fácil aprender milhões de recursos simples do que alguns complexos. Os identificadores dos documentos que estão sendo recuperados e as consultas canônicas não fornecem muita generalização, mas alinham sua classificação com os rótulos nas consultas principais. Portanto, não tenha medo de grupos de características em que cada uma se aplica a uma fração muito pequena dos seus dados, mas a cobertura geral é superior

a 90%. É possível usar a regularização para eliminar os recursos que se aplicam a poucos exemplos.

Regra 20: combine e modifique os recursos atuais para criar novos recursos de forma compreensível para humanos.

Há várias maneiras de combinar e modificar recursos. Sistemas de machine learning como o TensorFlow permitem pré-processar dados com transformações (https://www.tensorflow.org/tutorials/structured_data/preprocessing_layers?hl=pt-br). As duas abordagens mais comuns são "discretizações" e "cruzamentos".

A discretização consiste em pegar um recurso contínuo e criar muitos recursos discretos a partir dele. Considere um recurso contínuo, como a idade. Você pode criar um recurso que é 1 quando a idade é inferior a 18 anos, outro que é 1 quando a idade está entre 18 e 35 anos e assim por diante. Não pense demais nos limites desses histogramas: os quartis básicos vão dar a maior parte do impacto.

As interseções combinam duas ou mais colunas de atributos. Uma coluna de atributos, na terminologia do TensorFlow, é um conjunto de atributos homogêneos (por exemplo, {male, female}, {US, Canada, Mexico}, etc.). Uma interseção é uma nova coluna de atributos com atributos em, por exemplo, {male, female} × {US,Canada, Mexico}. Essa coluna de novos recursos vai conter o recurso (masculino, Canadá). Se você estiver usando o TensorFlow e pedir que ele crie esse cruzamento para você, esse recurso (masculino, Canadá) vai estar presente em exemplos que representam canadenses do sexo masculino. É necessário um grande volume de dados para aprender modelos com cruzamentos de três, quatro ou mais colunas de atributos básicos.

Cruzamentos que produzem colunas de elementos muito grandes podem ter ajuste excessivo. Por exemplo, imagine que você está fazendo algum tipo de pesquisa e tem uma coluna de atributos com palavras na consulta e outra com palavras no documento. É possível combinar esses elementos com uma cruz, mas você vai acabar com muitos recursos (consulte a Regra 21 (#rule_21_the_number_of_feature_weights_you_can_learn_in_a_linear_model_is_roughly_proportional_to_the_amount_of_data_you_have)
).

Ao trabalhar com texto, há duas alternativas. O mais draconiano é um produto escalar. Um produto de ponto na forma mais simples simplesmente conta o número de palavras em comum entre a consulta e o documento. Esse recurso pode ser discretizado. Outra

abordagem é uma interseção: assim, vamos ter um recurso presente se e somente se a palavra "pony" estiver no documento e na consulta, e outro recurso que estará presente se e somente se a palavra "the" estiver no documento e na consulta.

Regra 21: o número de pesos de recursos que você pode aprender em um modelo linear é aproximadamente proporcional à quantidade de dados que você tem.

Há resultados fascinantes da teoria de aprendizado estatístico sobre o nível adequado de complexidade para um modelo, mas essa regra é basicamente tudo o que você precisa saber. Já tive conversas em que as pessoas duvidavam que qualquer coisa pudesse ser aprendida com mil exemplos ou que você precisaria de mais de um milhão de exemplos porque elas ficam presas a um determinado método de aprendizado. O segredo é ajustar o aprendizado ao tamanho dos dados:

1. Se você estiver trabalhando em um sistema de classificação de pesquisa e houver milhões de palavras diferentes nos documentos e na consulta e você tiver 1.000 exemplos rotulados, use um produto de ponto entre os recursos de documento e consulta, TF-IDF (https://wikipedia.org/wiki/Tf%E2%80%93idf) e meia dúzia de outros recursos altamente projetados por humanos. 1.000 exemplos, uma dúzia de recursos.
2. Se você tiver um milhão de exemplos, cruze as colunas de atributos do documento e da consulta usando regularização e, possivelmente, seleção de atributos. Isso vai gerar milhões de recursos, mas com a regularização, você terá menos. Dez milhões de exemplos, talvez cem mil recursos.
3. Se você tiver bilhões ou centenas de bilhões de exemplos, poderá cruzar as colunas de recursos com tokens de documentos e consultas usando a seleção de recursos e a regularização. Você terá um bilhão de exemplos e 10 milhões de recursos. A teoria de aprendizado estatístico raramente fornece limites apertados, mas oferece ótimas orientações para um ponto de partida.

No final, use a Regra 28 (#rule_28_be_aware_that_identical_short_term_behavior_does_not_imply_identical_long_term_behavior) para decidir quais recursos usar.

Regra 22: limpe os recursos que você não usa mais.

Recursos não usados criam dívidas técnicas. Se você não estiver usando um recurso e a combinação dele com outros não estiver funcionando, remova-o da infraestrutura. Você quer

manter sua infraestrutura limpa para que os recursos mais promissores possam ser testados o mais rápido possível. Se necessário, alguém pode adicionar o recurso novamente.

Considere a cobertura ao decidir quais recursos adicionar ou manter. Quantos exemplos são cobertos pelo recurso? Por exemplo, se você tiver alguns recursos de personalização, mas apenas 8% dos seus usuários tiverem algum recurso de personalização, ele não será muito eficaz.

Ao mesmo tempo, alguns recursos podem ter um peso maior do que o esperado. Por exemplo, se você tiver um recurso que abrange apenas 1% dos dados, mas 90% dos exemplos que têm o recurso são positivos, ele será um ótimo recurso para adicionar.

Análise humana do sistema

Antes de passar para a terceira fase do aprendizado de máquina, é importante se concentrar em algo que não é ensinado em nenhuma aula de aprendizado de máquina: como analisar um modelo existente e melhorá-lo. Isso é mais uma arte do que uma ciência, e ainda assim há vários antipadrões que ajudam a evitá-la.

Regra 23: você não é um usuário final típico.

Essa é talvez a maneira mais fácil de uma equipe ficar presa. Embora haja muitos benefícios no fishfooding (usar um protótipo na sua equipe) e no dogfooding (usar um protótipo na sua empresa), os funcionários precisam verificar se o desempenho está correto. Embora uma mudança que seja obviamente ruim não deva ser usada, qualquer coisa que pareça razoavelmente próxima da produção precisa ser testada mais, seja pagando pessoas leigas para responder a perguntas em uma plataforma de crowdsourcing ou por meio de uma experiência em tempo real com usuários reais.

Há dois motivos para isso. A primeira é que você está muito perto do código. Talvez você esteja procurando um aspecto específico das postagens ou esteja simplesmente muito envolvido emocionalmente (por exemplo, viés de confirmação). A segunda é que seu tempo é muito valioso. Considere o custo de nove engenheiros em uma reunião de uma hora e pense em quantos rótulos humanos contratados são comprados em uma plataforma de crowdsourcing.

Se você realmente quer receber feedback do usuário, **use metodologias de experiência do usuário**. Crie perfis de usuário (uma descrição está no livro Sketching User Experiences

(https://play.google.com/store/books/details/Bill_Buxton_Sketching_User_Experiences_Getting_the?id=2vfPxocmLh0C&hl=pt-br)
, de Bill Buxton) no início de um processo e faça testes de usabilidade (uma descrição está no livro Don't Make Me Think
(https://play.google.com/store/books/details/Steve_Krug_Don_t_Make_Me_Think_Revisited?id=QlduAgAAQBAJ&hl=pt-br)
, de Steve Krug) mais tarde. Os perfis de usuário envolvem a criação de um usuário hipotético. Por exemplo, se a equipe for composta apenas por homens, pode ser útil criar uma persona de usuário do sexo feminino de 35 anos (com todos os recursos do usuário) e analisar os resultados gerados em vez de 10 resultados para homens de 25 a 40 anos. A inclusão de pessoas reais para observar a reação delas ao seu site (localmente ou remotamente) em testes de usabilidade também pode oferecer uma nova perspectiva.

Regra 24: meça a diferença entre os modelos.

Uma das medições mais fáceis e, às vezes, mais úteis que você pode fazer antes que os usuários analisem o novo modelo é calcular o quanto os novos resultados são diferentes da produção. Por exemplo, se você tiver um problema de classificação, execute os dois modelos em uma amostra de consultas em todo o sistema e analise o tamanho da diferença simétrica dos resultados (ponderada pela posição de classificação). Se a diferença for muito pequena, você poderá dizer sem executar um experimento que haverá poucas mudanças. Se a diferença for muito grande, verifique se a mudança é boa. Analisar as consultas em que a diferença simétrica é alta pode ajudar a entender qualitativamente como a mudança foi. No entanto, verifique se o sistema está estável. Verifique se um modelo, quando comparado a si mesmo, tem uma diferença simétrica baixa (idealmente zero).

Regra 25: ao escolher modelos, a performance utilitária supera o poder preditivo.

Seu modelo pode tentar prever a taxa de cliques. No entanto, no final, a questão principal é o que você faz com essa previsão. Se você estiver usando-o para classificar documentos, a qualidade da classificação final é mais importante do que a previsão em si. Se você prever a probabilidade de um documento ser spam e depois tiver um limite para o que é bloqueado, a precisão do que é permitido será mais importante. Na maioria das vezes, essas duas coisas precisam estar em concordância: quando não estão, é provável que haja um pequeno ganho. Portanto, se houver alguma mudança que melhore a perda de registro, mas degrade o desempenho do sistema, procure outro recurso. Quando isso começar a acontecer com mais frequência, é hora de revisar o objetivo do modelo.

Regra 26: procure padrões nos erros medidos e crie novos recursos.

Suponha que você veja um exemplo de treinamento que o modelo achou "errado". Em uma tarefa de classificação, esse erro pode ser um falso positivo ou um falso negativo. Em uma tarefa de classificação, o erro pode ser um par em que um positivo foi classificado como inferior a um negativo. O ponto mais importante é que este é um exemplo de que o sistema de aprendizado de máquina sabe que errou e gostaria de corrigir se tivesse a oportunidade. Se você fornecer um recurso que permita corrigir o erro, o modelo vai tentar usá-lo.

Por outro lado, se você tentar criar um recurso com base em exemplos que o sistema não considera como erros, ele será ignorado. Por exemplo, suponha que na Pesquisa de apps do Google Play, alguém pesquise "jogos sem custo financeiro". Suponha que um dos principais resultados seja um app de pegadinha menos relevante. Então, você cria um recurso para "apps de pegadinha". No entanto, se você estiver maximizando o número de instalações e as pessoas instalarem um app falso quando pesquisarem jogos sem custo financeiro, o recurso de "apps falsos" não terá o efeito desejado.

Depois de ter exemplos de erros do modelo, procure tendências que estejam fora do conjunto de recursos atual. Por exemplo, se o sistema parece estar rebaixando as postagens mais longas, adicione a duração da postagem. Não seja muito específico sobre os recursos que você adiciona. Se você for adicionar a duração do post, não tente adivinhar o que significa "longo". Adicione uma dúzia de recursos e deixe o modelo descobrir o que fazer com eles (consulte a Regra 21 (#rule_21_the_number_of_feature_weights_you_can_learn_in_a_linear_model_is_roughly_proportional_to_the_amount_of_data_you_have)). Essa é a maneira mais fácil de conseguir o que você quer.

Regra 27: tente quantificar o comportamento indesejável observado.

Alguns membros da sua equipe vão começar a ficar frustrados com as propriedades do sistema que não gostam e que não são capturadas pela função de perda atual. Nesse momento, eles precisam fazer o que for necessário para transformar as queixas em números sólidos. Por exemplo, se eles acharem que muitos "apps falsos" estão sendo mostrados na Pesquisa Google Play, eles podem pedir que avaliadores humanos identifiquem esses apps. É possível usar dados rotulados por humanos nesse caso, porque uma fração relativamente pequena das consultas representa uma fração grande do tráfego. Se os problemas forem mensuráveis, você poderá começar a usá-los como recursos, objetivos ou métricas. A regra geral é **medir primeiro, otimizar depois**.

Regra 28: comportamento idêntico no curto prazo não implica comportamento idêntico no longo prazo.

Imagine que você tem um novo sistema que analisa todos os doc_id e exact_query e calcula a probabilidade de clique para cada documento em cada consulta. Você descobre que o comportamento dele é quase idêntico ao seu sistema atual em testes side by side e A/B. Portanto, devido à simplicidade, você o lança. No entanto, você percebe que nenhum app novo está sendo mostrado. Por quê? Como seu sistema só mostra um documento com base no próprio histórico com essa consulta, não há como saber que um novo documento precisa ser mostrado.

A única maneira de entender como esse sistema funcionaria a longo prazo é treiná-lo apenas com dados adquiridos quando o modelo estava ativo. Isso é muito difícil.

Desvio de treinamento/disponibilização

O desvio de treinamento/disponibilização é uma diferença entre a performance durante o treinamento e a performance durante a disponibilização. Essa distorção pode ser causada por:

- Uma discrepância entre o modo como você manipula dados nos pipelines de treinamento e disponibilização.
- Uma mudança nos dados entre o treinamento e a veiculação.
- Um ciclo de feedback entre o modelo e o algoritmo.

Observamos sistemas de machine learning em produção no Google com distorção de treinamento- serviço que afeta negativamente o desempenho. A melhor solução é monitorar explicitamente para que as mudanças no sistema e nos dados não introduzam distorções sem serem notadas.

Regra 29: a melhor maneira de garantir que você treina como serve é salvar o conjunto de recursos usados no momento da veiculação e direcionar esses recursos para um registro para usá-los no momento do treinamento.

Mesmo que você não possa fazer isso para todos os exemplos, faça isso para uma pequena fração, para que possa verificar a consistência entre a veiculação e o treinamento (consulte a Regra 37 (#rule_37_measure_training_serving_skew)). As equipes que fizeram essa medição no Google às vezes ficaram surpresas com os resultados. A página inicial do YouTube (#youtube_overview) passou a registrar recursos no momento do envio com melhorias

significativas na qualidade e uma redução na complexidade do código. Muitas equipes estão mudando a infraestrutura.

Regra 30: não descarte dados amostrados com peso de importância de forma arbitrária.

Quando você tem muitos dados, é tentador usar os arquivos 1 a 12 e ignorar os arquivos 13 a 99. Isso não está certo. Embora os dados que nunca foram mostrados ao usuário possam ser descartados, a ponderação de importância é melhor para o resto. A ponderação de importância significa que, se você decidir que vai usar o exemplo X com uma probabilidade de 30%, atribua a ele um peso de 10/3. **Com a ponderação de importância, todas as propriedades de calibragem discutidas na Regra 14** (#rule_14_starting_with_an_interpretable_model_makes_debugging_easier) **ainda se aplicam.**

Regra 31: se você mesclar dados de uma tabela no momento do treinamento e da veiculação, os dados na tabela podem mudar.

Digamos que você combine IDs de documentos com uma tabela que contém recursos desses documentos, como o número de comentários ou cliques. Entre o treinamento e a veiculação, os recursos na tabela podem ser alterados. A previsão do modelo para o mesmo documento pode ser diferente entre o treinamento e a veiculação. A maneira mais fácil de evitar esse tipo de problema é registrar os recursos no momento da veiculação (consulte a Regra 32 (#rule_32_re_use_code_between_your_training_pipeline_and_your_serving_pipeline_whenever_possible)). Se a tabela estiver mudando lentamente, você também pode fazer um snapshot dela a cada hora ou dia para ter dados razoavelmente próximos. Isso ainda não resolve completamente o problema.

Regra 32: reutilize o código entre o pipeline de treinamento e o pipeline de veiculação sempre que possível.

O processamento em lote é diferente do processamento on-line. No processamento on-line, é necessário processar cada solicitação conforme ela chega (por exemplo, é necessário fazer uma pesquisa separada para cada consulta), enquanto no processamento em lote, é possível combinar tarefas (por exemplo, fazendo uma mesclagem). No momento da veiculação, você faz o processamento on-line, enquanto o treinamento é uma tarefa de processamento em lote. No entanto, há algumas coisas que você pode fazer para reutilizar o código. Por exemplo, é possível criar um objeto específico para seu sistema em que o resultado de consultas ou mesclagens possa ser armazenado de uma forma legível para humanos, e os erros podem ser testados com facilidade. Depois, quando você tiver coletado todas as informações, durante a

veiculação ou o treinamento, execute um método comum para fazer a ponte entre o objeto legível por humanos que é específico do seu sistema e qualquer formato que o sistema de machine learning espere. **Isso elimina uma fonte de desvio de treinamento/disponibilização**. Como consequência, tente não usar duas linguagens de programação diferentes entre o treinamento e a veiculação. Essa decisão vai tornar quase impossível compartilhar o código.

Regra 33: se você produzir um modelo com base nos dados até 5 de janeiro, teste o modelo com os dados de 6 de janeiro e depois.

Em geral, meça o desempenho de um modelo com os dados coletados após o treinamento, porque isso reflete melhor o que o sistema vai fazer na produção. Se você produzir um modelo com base nos dados até 5 de janeiro, teste o modelo com os dados de 6 de janeiro. Você espera que a performance não seja tão boa com os novos dados, mas não deve ser muito pior. Como pode haver efeitos diários, talvez você não consiga prever a taxa média de cliques ou de conversão, mas a área abaixo da curva, que representa a probabilidade de dar ao exemplo positivo uma pontuação maior do que um exemplo negativo, deve ser razoavelmente próxima.

Regra 34: na classificação binária para filtragem (como a detecção de spam ou a determinação de e-mails interessantes), faça pequenos sacrifícios de curto prazo na performance para ter dados muito limpos.

Em uma tarefa de filtragem, os exemplos marcados como negativos não são mostrados ao usuário. Digamos que você tenha um filtro que bloqueia 75% dos exemplos negativos na veiculação. Você pode querer extrair outros dados de treinamento das instâncias mostradas aos usuários. Por exemplo, se um usuário marcar um e-mail como spam que seu filtro deixou passar, você pode aprender com isso.

Mas essa abordagem introduz um viés de amostragem. Você pode coletar dados mais limpos se, durante a veiculação, você rotular 1% de todo o tráfego como "excluído" e enviar todos os exemplos excluídos para o usuário. Agora, seu filtro está bloqueando pelo menos 74% dos exemplos negativos. Esses exemplos podem se tornar seus dados de treinamento.

Se o filtro estiver bloqueando 95% dos exemplos negativos ou mais, essa abordagem se torna menos viável. Mesmo assim, se você quiser medir o desempenho da veiculação, é possível fazer uma amostra ainda menor (por exemplo, 0,1% ou 0,001%). Dez mil exemplos são suficientes para estimar a performance com bastante precisão.

Regra 35: tenha cuidado com a distorção inerente nos problemas de classificação.

Quando você muda o algoritmo de classificação de forma radical o suficiente para que resultados diferentes apareçam, você muda os dados que o algoritmo vai encontrar no futuro. Esse tipo de distorção vai aparecer, e você precisa projetar seu modelo em torno dela. Há várias abordagens diferentes. Essas abordagens são maneiras de favorecer dados que o modelo já conhece.

1. Ter uma maior regularização em recursos que abrangem mais consultas, em vez de recursos que estão ativados para apenas uma consulta. Dessa forma, o modelo vai favorecer recursos específicos para uma ou algumas consultas em vez de recursos que são generalizados para todas as consultas. Essa abordagem pode ajudar a evitar que resultados muito populares vazem para consultas irrelevantes. Isso é o oposto do conselho mais convencional de ter mais regularização nas colunas de recursos com mais valores exclusivos.
2. Permitir que apenas elementos tenham pesos positivos. Assim, qualquer recurso bom será melhor do que um recurso "desconhecido".
3. Não têm recursos exclusivos para documentos. Esta é uma versão extrema da #1. Por exemplo, mesmo que um app seja um download popular, independentemente da consulta, você não quer que ele seja mostrado em todos os lugares. Não ter recursos exclusivos de documento mantém isso simples. A razão pela qual você não quer mostrar um app popular específico em todos os lugares tem a ver com a importância de tornar todos os apps desejados acessíveis. Por exemplo, se alguém pesquisar "app de observação de pássaros", talvez faça o download de "Angry Birds", mas essa não era a intenção. Mostrar esse tipo de app pode melhorar a taxa de download, mas deixar as necessidades do usuário insatisfeitas.

Regra 36: evite loops de feedback com recursos posicionais.

A posição do conteúdo afeta drasticamente a probabilidade de interação do usuário com ele. Se você colocar um app na primeira posição, ele será clicado com mais frequência e você vai ter mais chances de clicar nele. Uma maneira de lidar com isso é adicionar recursos posicionais, ou seja, recursos sobre a posição do conteúdo na página. Você treina seu modelo com recursos posicionais, e ele aprende a ponderar, por exemplo, o recurso "1ª posição". Assim, seu modelo dá menos peso a outros fatores para exemplos com "1stposition=true". Em seguida, na veiculação, você não atribui o recurso de posição a nenhuma instância ou atribui a todas o mesmo recurso padrão, porque você está avaliando os candidatos antes de decidir a ordem em que eles serão exibidos.

É importante manter os atributos posicionais separados do resto do modelo devido a essa assimetria entre treinamento e teste. O ideal é que o modelo seja a soma de uma função dos elementos posicionais e uma função do restante dos elementos. Por exemplo, não cruze os recursos posicionais com nenhum recurso de documento.

Regra 37: medir o desvio de treinamento/disponibilização.

Há várias coisas que podem causar distorção no sentido mais geral. Além disso, é possível dividir o processo em várias partes:

- A diferença entre a performance nos dados de treinamento e nos dados de reserva. Em geral, isso sempre vai existir, e nem sempre é ruim.
- A diferença entre a performance nos dados de holdout e os dados de "dia seguinte". Novamente, isso sempre vai existir. Ajuste a regularização para maximizar o desempenho do dia seguinte. No entanto, grandes quedas no desempenho entre os dados de retenção e do dia seguinte podem indicar que alguns recursos são sensíveis ao tempo e podem estar prejudicando a performance do modelo.
- A diferença entre a performance nos dados do "dia seguinte" e os dados em tempo real. Se você aplicar um modelo a um exemplo nos dados de treinamento e ao mesmo exemplo no envio, ele vai gerar exatamente o mesmo resultado (consulte a Regra 5 (https://developers.google.com/machine-learning/guides/rules-of-ml/rule_5_test_the_infrastructure_independently_from_the_machine_learning?hl=pt-br)). Portanto, uma discrepância provavelmente indica um erro de engenharia.

Fase III do ML: crescimento lento, refinamento da otimização e modelos complexos

Haverá algumas indicações de que a segunda fase está chegando ao fim. Primeiro, seus ganhos mensais vão começar a diminuir. Você vai começar a ter compromises entre as métricas: algumas vão aumentar e outras vão diminuir em alguns experimentos. É aí que as coisas ficam interessantes. Como os ganhos são mais difíceis de alcançar, o aprendizado de máquina precisa ficar mais sofisticado. Observação: esta seção tem mais regras de céu azul do que as seções anteriores. Já vimos muitas equipes passarem pelos momentos felizes do aprendizado de máquina da Fase I e da Fase II. Quando a Fase III é alcançada, as equipes precisam encontrar o próprio caminho.

Regra 38: não perca tempo com novos recursos se os objetivos não alinhados se tornaram o problema.

À medida que as medições se estabilizam, a equipe começa a analisar problemas que estão fora do escopo dos objetivos do sistema de machine learning atual. Como mencionado anteriormente, se as metas do produto não forem cobertas pelo objetivo algorítmico atual, você precisará mudar o objetivo ou as metas do produto. Por exemplo, você pode otimizar cliques, adições ou downloads, mas tomar decisões de lançamento com base em parte dos avaliadores humanos.

Regra 39: as decisões de lançamento são um substituto para as metas de produto de longo prazo.

Alice tem uma ideia para reduzir a perda logística da previsão de instalações. Ela adiciona um recurso. A perda logística diminui. Quando ela faz um experimento ao vivo, ela percebe o aumento da taxa de instalação. No entanto, quando ela vai a uma reunião de análise de lançamento, alguém aponta que o número de usuários ativos por dia diminuiu 5%. A equipe decide não lançar o modelo. Alice fica desapontada, mas agora percebe que as decisões de lançamento dependem de vários critérios, e apenas alguns deles podem ser otimizados diretamente usando ML.

A verdade é que o mundo real não é Dungeons & Dragons: não há "pontos de impacto" que identifiquem a integridade do seu produto. A equipe precisa usar as estatísticas coletadas para tentar prever de maneira eficaz a qualidade do sistema no futuro. Eles precisam se preocupar com o engajamento, os usuários ativos por dia (DAU, na sigla em inglês), os usuários ativos em 30 dias, a receita e o retorno do investimento do anunciante. Essas métricas que podem ser medidas em testes A/B são apenas um substituto para metas de longo prazo, como satisfazer os usuários, aumentar o número de usuários, satisfazer os parceiros e ter lucro. Mesmo assim, você pode considerar substitutos para ter um produto útil e de alta qualidade e uma empresa próspera daqui a cinco anos.

As únicas decisões de lançamento fáceis são quando todas as métricas melhoram (ou pelo menos não pioram). Se a equipe tiver que escolher entre um algoritmo sofisticado de aprendizado de máquina e uma heurística simples, e se a heurística simples fizer um trabalho melhor em todas essas métricas, ela deve escolher a heurística. Além disso, não há uma classificação explícita de todos os valores de métricas possíveis. Especificamente, considere estes dois cenários:

Experimento	Usuários ativos por dia	Receita/dia
A	1 milhão	US$ 4 milhões
B	2 milhões	US$ 2 milhões

Se o sistema atual for A, é improvável que a equipe mude para B. Se o sistema atual for B, é improvável que a equipe mude para A. Isso parece estar em conflito com o comportamento racional. No entanto, as previsões de mudança de métricas podem ou não funcionar, e, portanto, há um grande risco envolvido em qualquer mudança. Cada métrica abrange algum risco que preocupa a equipe.

Além disso, nenhuma métrica aborda a principal preocupação da equipe, "onde meu produto vai estar daqui a cinco anos?"

Já as pessoas tendem a favorecer um objetivo que podem otimizar diretamente. A maioria das ferramentas de machine learning favorece esse ambiente. Um engenheiro que está desenvolvendo novos recursos pode ter um fluxo constante de lançamentos em um ambiente assim. Há um tipo de aprendizado de máquina, o aprendizado multiobjetivo, que começa a resolver esse problema. Por exemplo, é possível formular um problema de satisfação de restrição que tenha limites mais baixos em cada métrica e otimizar alguma combinação linear de métricas. No entanto, nem todas as métricas são facilmente enquadradas como objetivos de aprendizado de máquina: se um documento é clicado ou um app é instalado, é porque o conteúdo foi mostrado. No entanto, é muito mais difícil descobrir por que um usuário visita seu site. Como prever o sucesso futuro de um site como um todo é completo em IA (https://wikipedia.org/wiki/AI-complete): tão difícil quanto a visão computacional ou o processamento de linguagem natural.

Regra 40: mantenha os conjuntos simples.

Os modelos unificados que usam recursos brutos e classificam o conteúdo diretamente são os modelos mais fáceis de depurar e entender. No entanto, um conjunto de modelos (um "modelo" que combina as pontuações de outros modelos) pode funcionar melhor. **Para manter as coisas simples, cada modelo precisa ser um conjunto que considera apenas a entrada de outros modelos ou um modelo base que considera muitos recursos, mas não ambos.** Se você tiver modelos em cima de outros modelos treinados separadamente, a combinação deles pode resultar em um comportamento incorreto.

Use um modelo simples para agrupar que usa apenas a saída dos modelos "base" como entradas. Você também quer aplicar propriedades a esses modelos de conjunto. Por exemplo, um aumento na pontuação produzida por um modelo base não pode diminuir a pontuação do conjunto. Além disso, é melhor que os modelos recebidos sejam semanticamente interpretáveis (por exemplo, calibrados) para que as mudanças dos modelos subjacentes não confundam o modelo de conjunto. Além disso, **faça com que um aumento na probabilidade prevista de um classificador não diminua a probabilidade prevista do conjunto**.

Regra 41: quando a performance estagnar, procure novas fontes de informações qualitativas para adicionar em vez de refinar os indicadores atuais.

Você adicionou algumas informações demográficas sobre o usuário. Você adicionou algumas informações sobre as palavras no documento. Você fez a análise de modelos e ajustou a regularização. Você não teve um lançamento com mais de 1% de melhoria nas principais métricas em alguns trimestres. E agora?

É hora de começar a criar a infraestrutura para recursos radicalmente diferentes, como o histórico de documentos que esse usuário acessou no último dia, semana ou ano ou dados de uma propriedade diferente. Use entidades de wikidata (https://wikipedia.org/wiki/Wikidata) ou algo interno da sua empresa (como o Mapa de informações (https://wikipedia.org/wiki/Knowledge_Graph) do Google). Use a aprendizagem profunda. Comece a ajustar suas expectativas sobre o retorno esperado do investimento e amplie seus esforços de acordo com isso. Como em qualquer projeto de engenharia, é preciso ponderar o benefício de adicionar novos recursos em relação ao custo do aumento da complexidade.

Regra 42: não espere que a diversidade, a personalização ou a relevância estejam tão correlacionadas com a popularidade quanto você pensa.

A diversidade em um conjunto de conteúdo pode significar muitas coisas, sendo a diversidade da fonte do conteúdo uma das mais comuns. A personalização implica que cada usuário recebe os próprios resultados. A relevância implica que os resultados de uma consulta específica são mais adequados para essa consulta do que qualquer outra. Assim, as três propriedades são definidas como diferentes do normal.

O problema é que o comum tende a ser difícil de superar.

Se o sistema estiver medindo cliques, tempo gasto, visualizações, +1s, recompartilhamentos etc., você estará medindo a **popularidade** do conteúdo. Às vezes, as equipes tentam aprender um modelo pessoal com diversidade. Para personalizar, eles adicionam recursos que

permitiriam ao sistema personalizar (alguns recursos que representam o interesse do usuário) ou diversificar (recursos que indicam se esse documento tem algum recurso em comum com outros documentos retornados, como autor ou conteúdo), e descobrem que esses recursos recebem menos peso (ou às vezes um sinal diferente) do que o esperado.

Isso não significa que diversidade, personalização ou relevância não sejam valiosos. Conforme apontado na regra anterior, é possível fazer o pós-processamento para aumentar a diversidade ou a relevância. Se você notar um aumento nos objetivos de longo prazo, poderá declarar que a diversidade/relevância é valiosa, além da popularidade. Você pode continuar usando o pós-processamento ou modificar diretamente o objetivo com base na diversidade ou na relevância.

Regra 43: seus amigos tendem a ser os mesmos em produtos diferentes. Seus interesses tendem a não ser.

As equipes do Google tiveram muito sucesso ao usar um modelo que previa a proximidade de uma conexão em um produto e o fazia funcionar bem em outro. Seus amigos são quem são. Por outro lado, observei várias equipes com dificuldades com os recursos de personalização em diferentes produtos. Sim, parece que vai funcionar. Por enquanto, não parece que sim. O que às vezes funciona é usar dados brutos de uma propriedade para prever o comportamento em outra. Além disso, lembre-se de que saber que um usuário tem um histórico em outra propriedade pode ajudar. Por exemplo, a presença de atividade do usuário em dois produtos pode ser indicativa por si só.

Trabalhos relacionados

Há muitos documentos sobre aprendizado de máquina no Google e externamente.

- Curso intensivo de aprendizado de máquina (https://developers.google.com/machine-learning/crash-course?hl=pt-br): uma introdução ao aprendizado de máquina aplicado.
- Machine Learning: A Probabilistic Approach (https://www.cs.ubc.ca/%7Emurphyk/MLbook/) (em inglês) de Kevin Murphy para entender o campo do aprendizado de máquina.
- Boa análise de dados (https://developers.google.com/machine-learning/guides/good-data-analysis?hl=pt-br): uma abordagem de ciência de dados para pensar sobre conjuntos de dados.

- Deep Learning (http://www.iro.umontreal.ca/%7Ebengioy/dlbook/) (link em inglês) de Ian Goodfellow e outros para aprender modelos não lineares.
- Documento do Google sobre dívida técnica (http://research.google.com/pubs/pub43146.html?hl=pt-br), que tem vários conselhos gerais.
- Documentação do TensorFlow (https://www.tensorflow.org/?hl=pt-br).

Agradecimentos

Agradecemos a David Westbrook, Peter Brandt, Samuel Ieong, Chenyu Zhao, Li Wei, Michalis Potamias, Evan Rosen, Barry Rosenberg, Christine Robson, James Pine, Tal Shaked, Tushar Chandra, Mustafa Ispir, Jeremiah Harmsen, Konstantinos Katsiapis, Glen Anderson, Dan Duckworth, Shishir Birmiwal, Gal Elidan, Su Lin Wu, Jaihui Liu, Fernando Pereira e Hrishikesh Aradhye pelas muitas correções, sugestões e exemplos úteis para este documento. Agradecemos também a Kristen Lefevre, Suddha Basu e Chris Berg, que ajudaram com uma versão anterior. Todos os erros, omissões ou (surpresa!) opiniões impopulares são meus.

Apêndice

Este documento contém várias referências a produtos do Google. Para fornecer mais contexto, confira uma breve descrição dos exemplos mais comuns abaixo.

Visão geral do YouTube

O YouTube é um serviço de streaming de vídeo. As equipes do YouTube Assistir a seguir e da página inicial do YouTube usam modelos de ML para classificar as recomendações de vídeo. A seção "Próximo" recomenda vídeos para assistir depois do que está sendo reproduzido no momento, enquanto a página inicial recomenda vídeos para os usuários que estão navegando na página inicial.

Visão geral do Google Play

O Google Play tem muitos modelos que resolvem uma variedade de problemas. A Pesquisa Google Play, as recomendações personalizadas da página inicial da Play Store e os apps "Usuários também instalaram" usam machine learning.

Visão geral do Google+

O Google Plus usava aprendizado de máquina em várias situações: classificar postagens na "timeline" que o usuário acessa, classificar postagens "What's Hot" (postagens que são muito populares no momento), classificar pessoas que você conhece e assim por diante. O Google Plus fechou todas as contas pessoais em 2019 e foi substituído pelo Google Currents para contas empresariais em 6 de julho de 2020.

Exceto em caso de indicação contrária, o conteúdo desta página é licenciado de acordo com a Licença de atribuição 4.0 do Creative Commons (https://creativecommons.org/licenses/by/4.0/), e as amostras de código são licenciadas de acordo com a Licença Apache 2.0 (https://www.apache.org/licenses/LICENSE-2.0). Para mais detalhes, consulte as políticas do site do Google Developers (https://developers.google.com/site-policies?hl=pt-br). Java é uma marca registrada da Oracle e/ou afiliadas.

Última atualização 2025-01-13 UTC.